Subarray Sort

AlgoExpert · Hard · Arrays

The problem

- Smallest [start, end] range which, if sorted, makes the whole array sorted; [-1, -1] if sorted.

Intuition

- Find the smallest and largest values that are out of order; their correct positions bound the subarray.

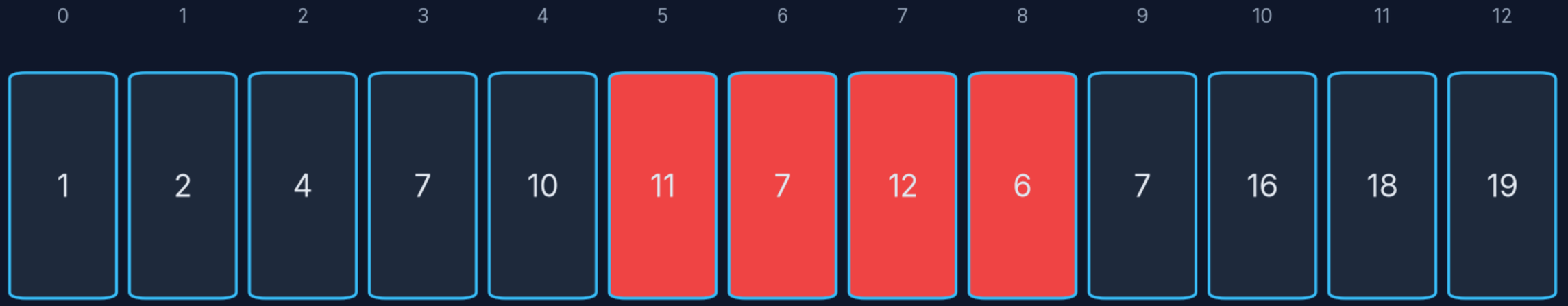
Brute force vs optimal

- Sort a copy and compare → O(n log n)
- Find min/max out of place, then their positions → O(n)
- O(1) space

Approach

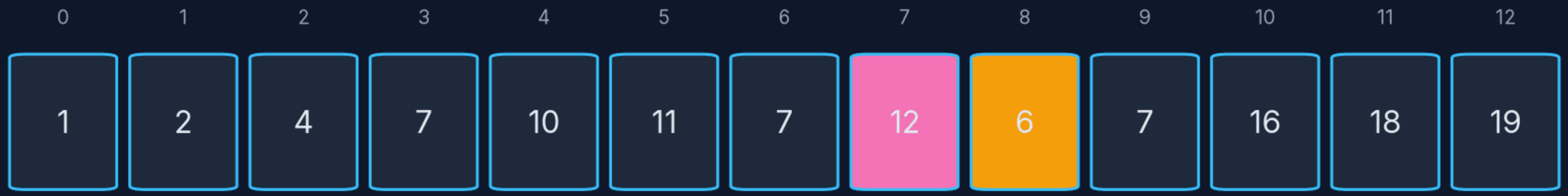
- 1. Scan for elements out of order with a neighbour
- 2. Track the min and max among them
- 3. Find where min belongs from the left and max from the right

Worked example



Find out-of-order elements

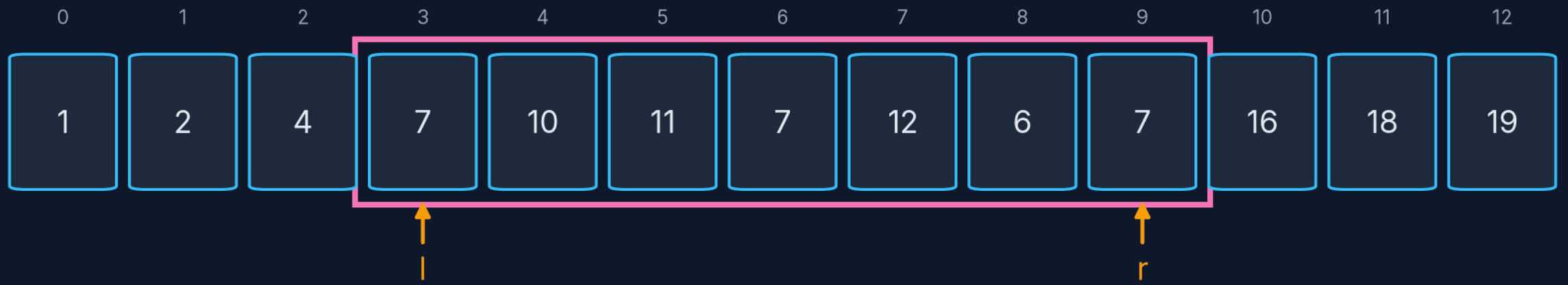
Worked example




```
min out of order = 6
max out of order = 12
```

Smallest = 6, largest = 12

Worked example



6 belongs at index 3; 12 at index 9

Answer: [3, 9]

Java solution

```java
public class SubarraySort {
    public static int[] subarraySort(int[] a) {
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
        for (int i = 0; i < a.length; i++) {
            boolean out = (i > 0 && a[i] < a[i - 1]) || (i < a.length - 1 && a[i] > a[i + 1]);
            if (out) { min = Math.min(min, a[i]); max = Math.max(max, a[i]); }
        }
        if (min == Integer.MAX_VALUE) return new int[]{-1, -1};
        int l = 0, r = a.length - 1;
        while (a[l] <= min) l++;
        while (a[r] >= max) r--;
        return new int[]{l, r};
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Already sorted → [−1, −1]
- Duplicates: use ≤ / ≥
- Out-of-order values may be far from the boundary

Interview tips & follow-ups

- Explain why min/max determine the bounds

Recap

- Find the smallest and largest values that are out of order; their correct positions bound the subarray.
- Complexity: O(n)